

UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

DIPARTIMENTO DI INFORMATICA, SISTEMISTICA E
COMUNICAZIONE

CORSO DI LAUREA MAGISTRALE IN INFORMATICA



Archittura Dati

Valutazione dell'Impatto del Rumore sulla Predizione
dei Modelli di Machine Learning

Autore:

Alen Bicanic Matr. 945230

Samuele Perotti Matr. 899817

Anno Accademico 2025-2026

Indice

1	Introduzione	2
2	Descrizione del dataset	2
3	Preprocessing dei dati	3
3.1	Rimozione di colonne non informative	3
3.2	Conversione del target in problema binario	3
3.3	Gestione delle variabili booleane	3
3.4	One-hot encoding delle variabili categoriche	3
3.5	Gestione dei valori mancanti residui	4
4	Analisi esplorativa (EDA)	4
4.1	Distribuzione del target binario	4
4.2	Matrice di correlazione	5
4.3	Distribuzione delle variabili numeriche	5
5	Suddivisione train/test	5
6	Modelli di baseline	6
6.1	Decision Tree	6
6.2	Neural Network (Multi-Layer Perceptron)	8
6.3	Support Vector Machine (SVM)	10
6.4	Confronto tra i modelli di baseline	11
7	Introduzione del rumore nel dataset	12
7.1	Valori mancanti (NaN)	12
7.2	Valori duplicati	12
7.3	Dati "sporchi"	12
8	Impatto del rumore sulle prestazioni dei modelli	13
8.1	Valori mancanti	13
8.2	Valori duplicati	14
8.3	Dati sporchi	15
9	Tecniche di preprocessing e regolarizzazione per la mitigazione del rumore	15
9.1	Mitigazione dei valori mancanti	15
9.2	Mitigazione dei valori duplicati	17
9.3	Mitigazione dei dati sporchi	18
10	Confronto finale tra le strategie	19
11	Conclusioni	20
A	Elenco delle immagini da inserire manualmente	21

1 Introduzione

L'obiettivo del progetto è comprendere in che modo la presenza di rumore nei dati influenzi le prestazioni dei modelli di Machine Learning e individuare le strategie più efficaci per limitarne l'impatto. Il lavoro si è articolato nelle seguenti fasi, in linea con la proposta progettuale concordata:

- scelta e analisi di un dataset adatto al problema di classificazione;
- preparazione dei dati e creazione del training set;
- introduzione di rumore nel dataset a diversi livelli di intensità;
- addestramento di più modelli sul dataset modificato e analisi delle prestazioni al variare del rumore;
- applicazione di tecniche di preprocessing e regolarizzazione (normalizzazione/scaling, PCA, cross-validation, early stopping, dropout, imputazione avanzata, gestione dei duplicati) per valutarne l'efficacia nel contenere il degrado delle prestazioni;
- confronto tra i risultati ottenuti sui dataset rumorosi e quelli ottenuti sul dataset originale;
- valutazione delle performance mediante le metriche di accuracy, precision e recall.

Il dominio applicativo scelto è quello medicale/di supporto alla diagnosi: si è utilizzato l'*UCI Heart Disease Dataset*, con l'obiettivo di prevedere la presenza o assenza di una malattia cardiaca a partire da variabili cliniche (classificazione binaria).

2 Descrizione del dataset

Il dataset utilizzato è lo **UCI Heart Disease Dataset**, reso disponibile su Kaggle¹ e caricato in questo progetto da un repository GitHub del corso. Il dataset è composto da **920 istanze** e, nella sua forma originale, da **16 colonne**, tra cui variabili demografiche (età, sesso), cliniche (pressione a riposo, colesterolo, frequenza cardiaca massima, angina da sforzo, ecc.) e la variabile target **num**.

La variabile target originale è **multiclasse** (valori da 0 a 4, dove 0 indica assenza di malattia e i valori da 1 a 4 indicano livelli crescenti di gravità). La distribuzione originale delle classi è riportata in Tabella 1.

¹<https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data>

Classe (num)	Numero di istanze
0 (assenza)	411
1	265
2	109
3	107
4	28

Tabella 1: Distribuzione originale della variabile target multiclasse.

Diverse variabili presentano valori mancanti in misura significativa (ad esempio la colonna `ca`, presente solo in 309 istanze su 920), aspetto che ha reso necessaria un’attenta fase di preprocessing, descritta nella sezione seguente.

3 Preprocessing dei dati

Prima dell’addestramento dei modelli, il dataset è stato sottoposto alle seguenti operazioni di pulizia e trasformazione.

3.1 Rimozione di colonne non informative

Le colonne `id` (identificativo univoco della riga) e `dataset` (indicante solo la provenienza geografica del campione) sono state rimosse in quanto non contribuiscono alla predizione.

3.2 Conversione del target in problema binario

Per semplicità e coerenza clinica, il target multiclasse `num` è stato trasformato in una variabile binaria `target`:

- 0 = assenza di malattia;
- 1 = presenza di malattia (aggregazione delle classi originarie 1–4).

3.3 Gestione delle variabili booleane

Le variabili `fbs` (glicemia a digiuno) ed `exang` (angina indotta da sforzo) sono state convertite in formato numerico 0/1, imputando i valori mancanti con `False` prima della conversione.

3.4 One-hot encoding delle variabili categoriche

Le variabili categoriche `sex`, `cp`, `restecg`, `slope` e `thal` sono state trasformate tramite *one-hot encoding* (con rimozione della prima categoria per evitare collinearità, `drop_first=True`).

3.5 Gestione dei valori mancanti residui

Dopo l'encoding, le colonne numeriche che presentavano ancora valori mancanti (`trestbps`, `chol`, `thalch`, `oldpeak`, `ca`) sono state imputate con la **mediana** della rispettiva colonna. Questa scelta è stata preferita alla media in quanto più robusta rispetto agli outlier e permette di non perdere istanze, aspetto particolarmente rilevante data la dimensione limitata del dataset. Il conteggio dei valori mancanti per colonna, riscontrato dopo l'encoding, è riportato in Tabella 2.

Colonna	Valori mancanti
<code>trestbps</code>	59
<code>chol</code>	30
<code>thalch</code>	55
<code>oldpeak</code>	62
<code>ca</code>	611

Tabella 2: Valori mancanti per colonna numerica prima dell'imputazione.

4 Analisi esplorativa (EDA)

Dopo il preprocessing è stata condotta un'analisi esplorativa per comprendere la struttura del dataset ripulito.

4.1 Distribuzione del target binario

Il grafico a barre della distribuzione del target binario mostra un discreto bilanciamento tra le due classi (assenza/presenza di malattia), condizione favorevole per l'addestramento dei modelli di classificazione.

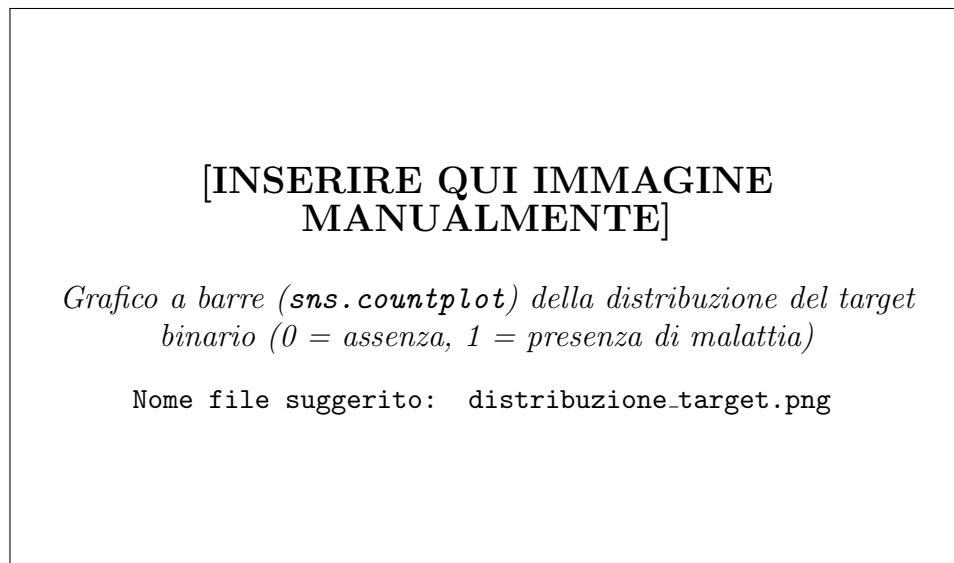


Figura 1: Distribuzione della variabile target binaria

4.2 Matrice di correlazione

La matrice di correlazione tra tutte le variabili numeriche del dataset permette di individuare eventuali relazioni lineari forti tra le feature e con il target.

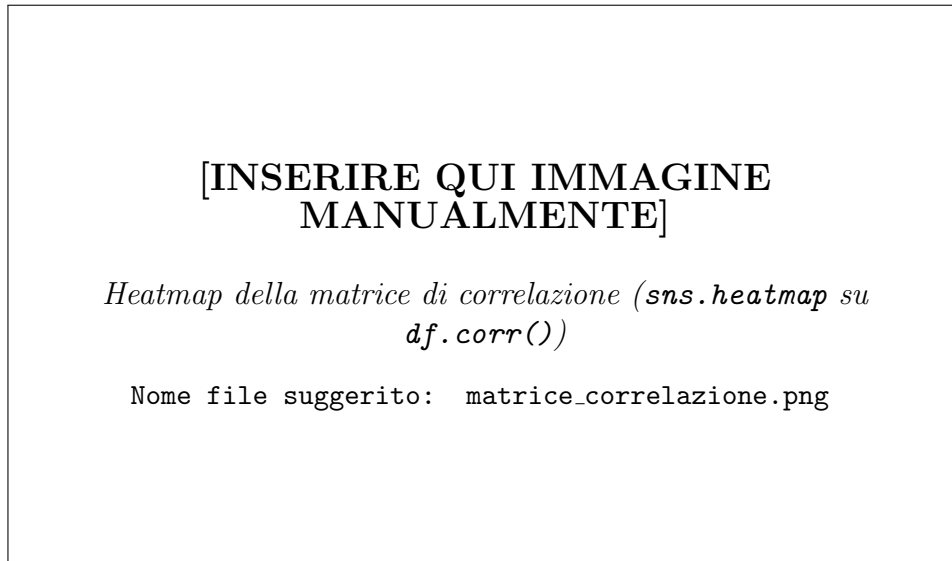


Figura 2: Matrice di correlazione tra le variabili del dataset

4.3 Distribuzione delle variabili numeriche

Gli istogrammi delle variabili numeriche permettono di valutarne la forma distributiva e la presenza di eventuali asimmetrie o outlier.

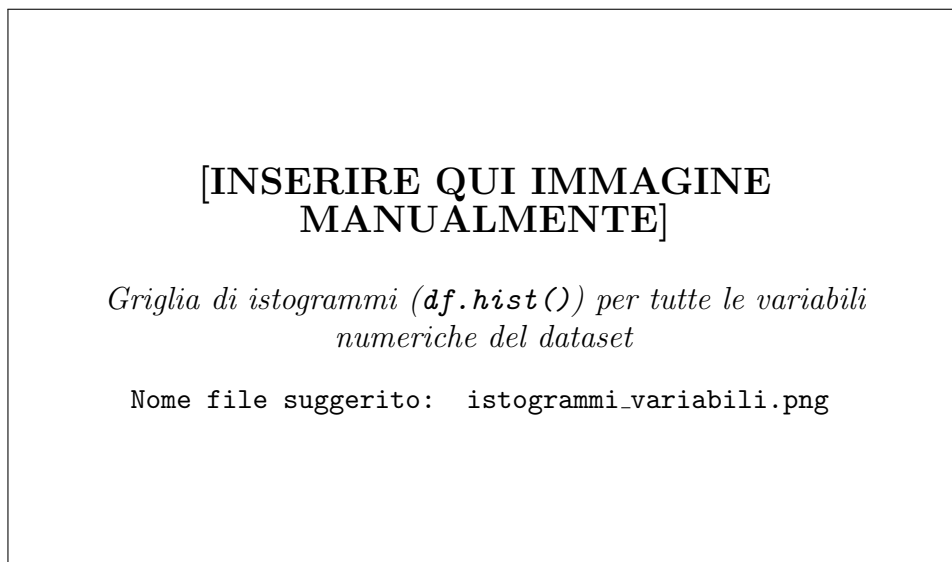


Figura 3: Istogrammi delle variabili numeriche

5 Suddivisione train/test

Il dataset è stato suddiviso in training set e test set con proporzione 80/20 (`test_size=0.2`), utilizzando la stratificazione sulla variabile target (`stratify=y`)

per preservare le proporzioni delle classi in entrambi i sottoinsiemi, e fissando `random_state=42` per la riproducibilità dei risultati.

6 Modelli di baseline

Sono stati addestrati e confrontati tre modelli di classificazione supervisionata sul dataset originale (privo di rumore aggiuntivo).

6.1 Decision Tree

È stato inizialmente addestrato un Decision Tree con criterio *entropy* senza vincoli di profondità, ottenendo i seguenti risultati sul test set:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.73	0.77	0.75
Recall	0.71	0.79	0.75
F1-score	0.72	0.78	0.75

Tabella 3: Decision Tree (non potato) – Accuracy sul test set: 0.7554



Figura 4: Matrice di confusione – Decision Tree (baseline)

[INSERIRE QUI IMMAGINE
MANUALMENTE]

*Visualizzazione grafica dell'albero di decisione non potato
(`plot_tree`)*

Nome file suggerito: `albero_non_potato.png`

Figura 5: Albero di decisione non potato

L'albero non potato tende a sovradattare (*overfitting*) i dati di training. Per contenere questo effetto è stato introdotto un vincolo sulla profondità massima (`max_depth=5`), ottenendo un modello più semplice, più interpretabile e con prestazioni migliori sul test set:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.80	0.81	0.80
Recall	0.74	0.85	0.80
F1-score	0.77	0.83	0.80

Tabella 4: Decision Tree potato (`max_depth=5`) – Accuracy sul test set: 0.8043

[INSERIRE QUI IMMAGINE
MANUALMENTE]

Matrice di confusione del Decision Tree potato sul test set

Nome file suggerito: `cm.dt.pruned.png`

Figura 6: Matrice di confusione – Decision Tree (potato)

[INSERIRE QUI IMMAGINE
MANUALMENTE]

*Visualizzazione grafica dell'albero di decisione potato con
max_depth=5*

Nome file suggerito: albero_potato.png

Figura 7: Albero di decisione potato

La validazione tramite 5-fold cross-validation sul Decision Tree potato ha restituito un'accuracy media pari a **0.725**, coerente con quanto osservato sul singolo test set.

6.2 Neural Network (Multi-Layer Perceptron)

È stata definita una rete neurale feed-forward per la classificazione binaria, con la seguente architettura:

- Strato di input + primo hidden layer: 16 neuroni, attivazione sigmoide;
- Secondo hidden layer: 8 neuroni, attivazione sigmoide;
- Strato di output: 1 neurone, attivazione sigmoide (adatta alla classificazione binaria).

Il modello è stato compilato con funzione di perdita `binary_crossentropy` e ottimizzatore `adam`, e addestrato per 100 epoche con `batch_size=16` sui dati precedentemente standardizzati (`StandardScaler`), passaggio necessario poiché le reti neurali sono sensibili alla scala delle feature in input.

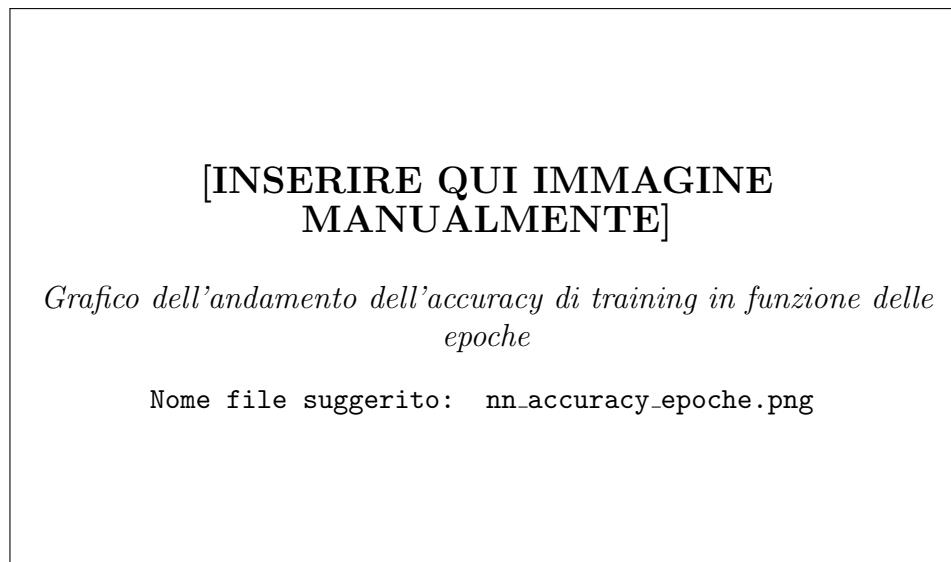


Figura 8: Andamento dell'accuracy di training (Neural Network)

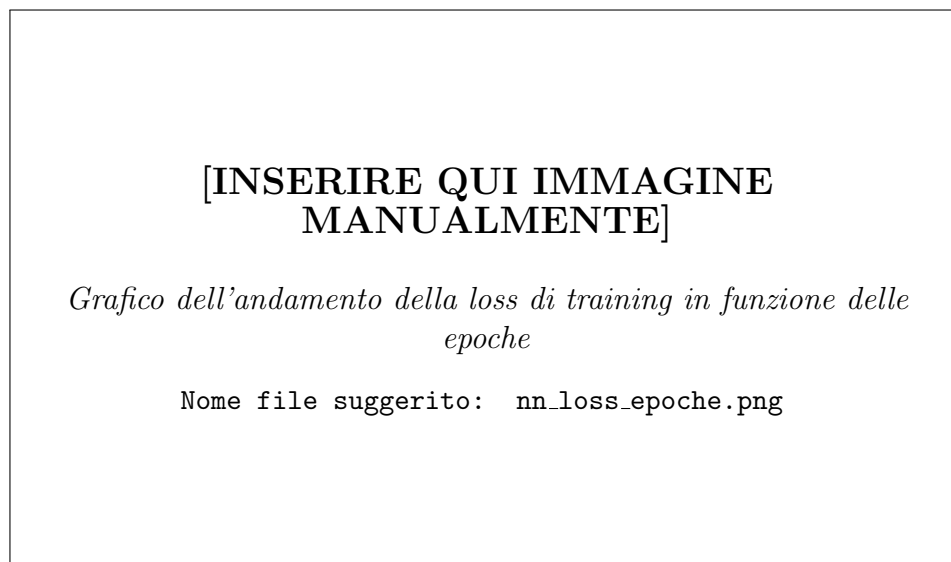


Figura 9: Andamento della loss di training (Neural Network)

Sul test set, il modello ha ottenuto una **Test Accuracy di 0.8315** (Test Loss = 0.3813), con le seguenti metriche di classificazione:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.83	0.83	0.83
Recall	0.78	0.87	0.83
F1-score	0.81	0.85	0.83

Tabella 5: Neural Network – Accuracy sul test set: 0.8315

[INSERIRE QUI IMMAGINE
MANUALMENTE]

Matrice di confusione della rete neurale sul test set

Nome file suggerito: `cm_nn_baseline.png`

Figura 10: Matrice di confusione – Neural Network

6.3 Support Vector Machine (SVM)

Per il terzo modello è stata scelta una SVM con le seguenti motivazioni:

- **Kernel RBF (Radial Basis Function):** a differenza di un kernel lineare, il kernel RBF gestisce relazioni non lineari tra le feature; dato che le variabili cliniche (pressione, colesterolo, età, ecc.) spesso interagiscono in modo complesso, un confine di decisione non lineare tende a offrire prestazioni migliori;
- **Parametro di regolarizzazione $C = 1.0$:** controlla il compromesso (*trade-off*) tra l'ampiezza del margine di separazione e la corretta classificazione dei punti di training.

Il modello, addestrato sui dati standardizzati, ha ottenuto un'**Accuracy di 0.8424** sul test set:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.90	0.81	0.85
Recall	0.73	0.93	0.84
F1-score	0.81	0.87	0.84

Tabella 6: SVM (kernel RBF, $C=1.0$) – Accuracy sul test set: 0.8424

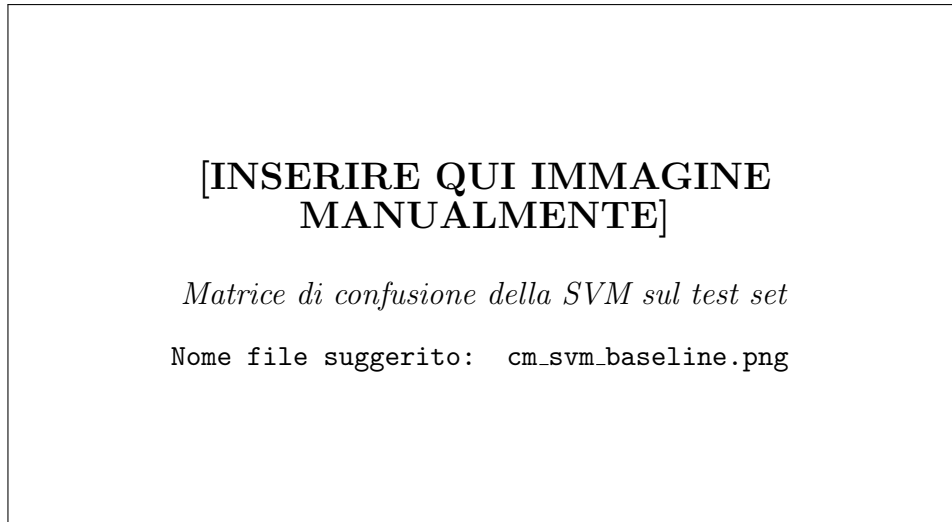


Figura 11: Matrice di confusione – SVM

6.4 Confronto tra i modelli di baseline

La Tabella 7 riassume le prestazioni dei quattro modelli addestrati (Decision Tree non potato, Decision Tree potato, Neural Network, SVM) sul dataset originale, privo di rumore.

Modello	Accuracy	Precision	Recall	F1-score
Decision Tree (baseline)	0.755	0.75	0.75	0.75
Decision Tree (potato)	0.804	0.80	0.80	0.80
Neural Network	0.832	0.83	0.83	0.83
SVM	0.842	0.85	0.84	0.84

Tabella 7: Confronto delle metriche sul test set tra i modelli di baseline (valori pesati/medi).

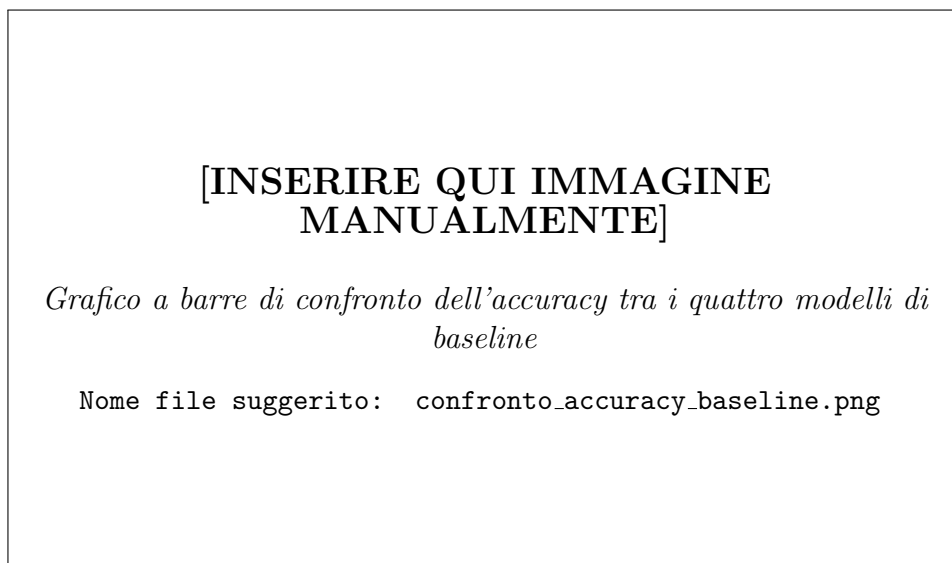


Figura 12: Confronto dell'Accuracy tra i modelli di baseline

La SVM con kernel RBF risulta il modello con le prestazioni migliori sul test set, seguita dalla rete neurale; il Decision Tree potato mostra un netto miglioramento rispetto alla versione non potato, a conferma dell'efficacia del pruning nel ridurre l'overfitting.

7 Introduzione del rumore nel dataset

Per valutare la robustezza dei modelli, è stato introdotto rumore artificiale nel training set (il test set rimane sempre pulito, per garantire un confronto equo tra le diverse condizioni), a tre livelli di intensità crescente:

Livello	Percentuale di rumore
Low	15%
Medium	40%
High	55%

Tabella 8: Livelli di rumore utilizzati negli esperimenti.

Sono state simulate tre diverse tipologie di rumore, ciascuna rappresentativa di problematiche comuni nei dataset reali.

7.1 Valori mancanti (NaN)

È stata implementata una funzione che introduce valori NaN in modo casuale nel training set, generando per ogni cella un numero casuale e sostituendo il valore originale con NaN se tale numero è inferiore alla soglia di rumore desiderata. Il rumore è stato applicato *esclusivamente* su `X_train`, in modo da poter confrontare in modo equo le prestazioni sul medesimo test set pulito. La percentuale effettiva di valori mancanti introdotti è risultata coerente con quella target (15.2%, 40.4%, 55.7% rispettivamente).

7.2 Valori duplicati

È stata implementata una funzione che duplica una percentuale di righe del training set, scelte casualmente (con reinserimento), simulando errori di raccolta/unione dati che generano osservazioni ripetute. Le dimensioni del training set risultante sono riportate in Tabella 9.

Livello	Dimensione training set	Dimensione originale
Low (15%)	846	736
Medium (40%)	1030	736
High (55%)	1140	736

Tabella 9: Dimensione del training set dopo l'inserimento di duplicati.

7.3 Dati "sporchi"

È stata implementata una funzione che introduce rumore realistico differenziato in base al tipo di variabile:

- sulle colonne **continue**, viene aggiunto rumore gaussiano proporzionale alla deviazione standard della colonna stessa, applicato solo a una frazione casuale di righe (in base al livello di rumore);
- sulle colonne **binarie** (incluse quelle ottenute da one-hot encoding), viene effettuato uno *swap* dei valori (0 diventa 1 e viceversa) su una frazione casuale di righe.

Questa tipologia di rumore simula errori di misurazione/inserimento tipici dei dati reali, più insidiosi dei semplici valori mancanti perché non facilmente identificabili.

8 Impatto del rumore sulle prestazioni dei modelli

Per ciascuna tipologia di rumore, i tre modelli (Decision Tree, SVM, Neural Network) sono stati riaddestrati sul training set corrotto e valutati sul medesimo test set pulito, applicando un preprocessing di base (imputazione con mediana per i NaN, scaling standard). Le prestazioni sono state inoltre validate tramite 5-fold cross-validation stratificata (`StratifiedKFold`, `k=5`) sul training set corrotto.

8.1 Valori mancanti

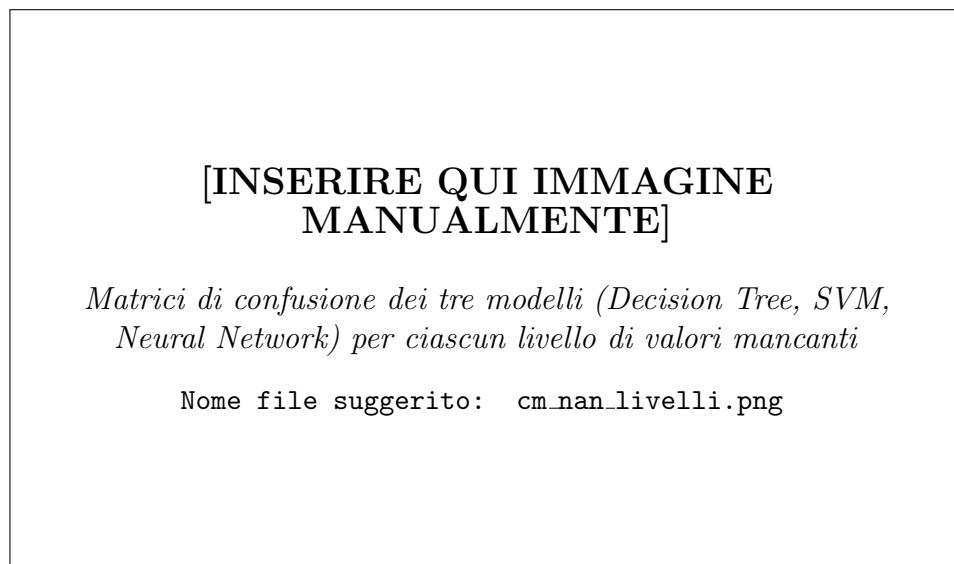


Figura 13: Matrici di confusione al variare del livello di valori mancanti

La Tabella 10 riporta l'accuracy media di cross-validation al variare del livello di valori mancanti, con preprocessing di base (imputazione mediana + scaling).

Livello	Decision Tree	SVM	Neural Network
Baseline (0%)	0.754	0.818	0.802
Low (15%)	0.730	0.798	0.792
Medium (40%)	0.704	0.761	0.777
High (55%)	0.716	0.693	0.743

Tabella 10: Accuracy media di CV (5-fold) al variare del livello di valori mancanti, con imputazione base (mediana).

Si osserva un degrado progressivo delle prestazioni all'aumentare della percentuale di valori mancanti, particolarmente marcato per la SVM, che passa da 0.818 (baseline) a 0.693 (55% di rumore).

8.2 Valori duplicati

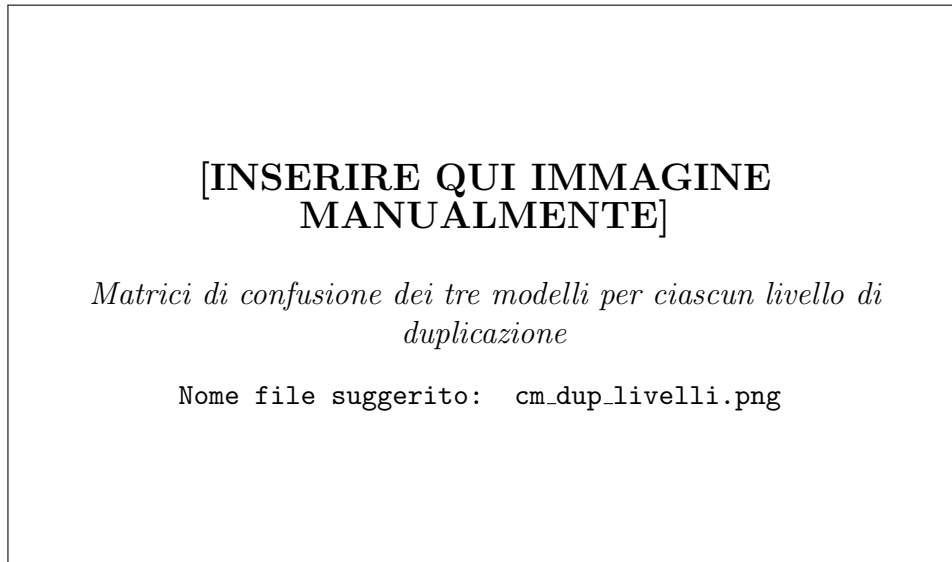


Figura 14: Matrici di confusione al variare del livello di duplicazione

Livello	Decision Tree	SVM	Neural Network
Baseline (0%)	0.754	0.818	0.807
Low (15%)	0.784	0.830	0.811
Medium (40%)	0.777	0.847	0.824
High (55%)	0.799	0.854	0.823

Tabella 11: Accuracy media di CV (5-fold) al variare del livello di duplicazione (dati non ripuliti).

A differenza delle altre tipologie di rumore, l'accuracy in cross-validation **aumenta** apparentemente con il livello di duplicazione. Questo risultato è **fuorviante**: è dovuto a un fenomeno di *data leakage*, poiché la suddivisione in fold della cross-validation può separare copie identiche della stessa riga tra training e validation del fold, rendendo la stima della performance eccessivamente ottimistica. Questo aspetto viene analizzato e corretto nella Sezione 9.2.

8.3 Dati sporchi

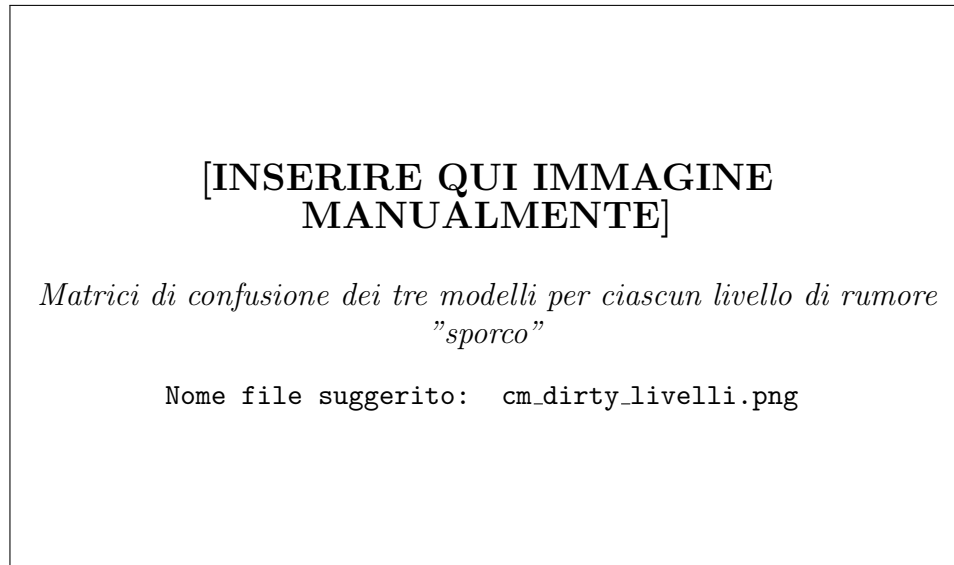


Figura 15: Matrici di confusione al variare del livello di dati sporchi

Livello	Decision Tree	SVM	Neural Network
Baseline (0%)	0.754	0.818	0.804
Low (15%)	0.731	0.766	0.760
Medium (40%)	0.693	0.683	0.686
High (55%)	0.647	0.697	0.709

Tabella 12: Accuracy media di CV (5-fold) al variare del livello di dati sporchi, con modelli non regolarizzati.

Questa tipologia di rumore risulta la più dannosa per le prestazioni: tutti e tre i modelli mostrano un calo consistente e monotono dell'accuracy all'aumentare del livello di rumore, poiché i valori errati (ma plausibili) alterano direttamente la relazione tra feature e target senza essere facilmente rilevabili come nel caso dei NaN.

9 Tecniche di preprocessing e regolarizzazione per la mitigazione del rumore

Per ciascuna tipologia di rumore sono state applicate tecniche specifiche di preprocessing e/o regolarizzazione dei modelli, al fine di valutarne l'efficacia nel contenere il degrado delle prestazioni osservato nella sezione precedente.

9.1 Mitigazione dei valori mancanti

Oltre all'imputazione di base con la mediana, sono state testate due strategie di imputazione più avanzate, in combinazione con tecniche di regolarizzazione dei modelli (Decision Tree con `max_depth=4`, `min_samples_split=20`,

`min_samples_leaf=15`; SVM con margine più "morbido", $C = 0.1$; rete neurale con Dropout (0.3 e 0.2) e EarlyStopping):

- **KNN Imputer**: imputa ciascun valore mancante sulla base della media dei $k = 5$ vicini più simili;
- **Iterative Imputer**: stima ogni valore mancante modellandolo come funzione delle altre variabili, in modo iterativo (`max_iter=10`).

La Tabella 13 confronta l'accuracy media di cross-validation ottenuta con le tre strategie di imputazione (per la SVM, unico modello riportato a titolo di confronto sintetico – i risultati completi per tutti i modelli sono disponibili nel notebook).

Livello	SimpleImputer (base)	KNN Imputer	Iterative Imputer
Baseline (0%)	0.818	0.823	0.823
Low (15%)	0.798	0.799	0.795
Medium (40%)	0.761	0.768	0.766
High (55%)	0.693	0.723	0.739

Tabella 13: Accuracy media di CV (SVM) al variare della strategia di imputazione dei valori mancanti.

Le strategie di imputazione avanzata (KNN e Iterative Imputer), unite alla regolarizzazione dei modelli, mostrano un beneficio soprattutto ai livelli di rumore più elevati (Medium e High), dove il recupero di informazione tramite le relazioni tra variabili risulta più efficace della semplice sostituzione con la mediana.

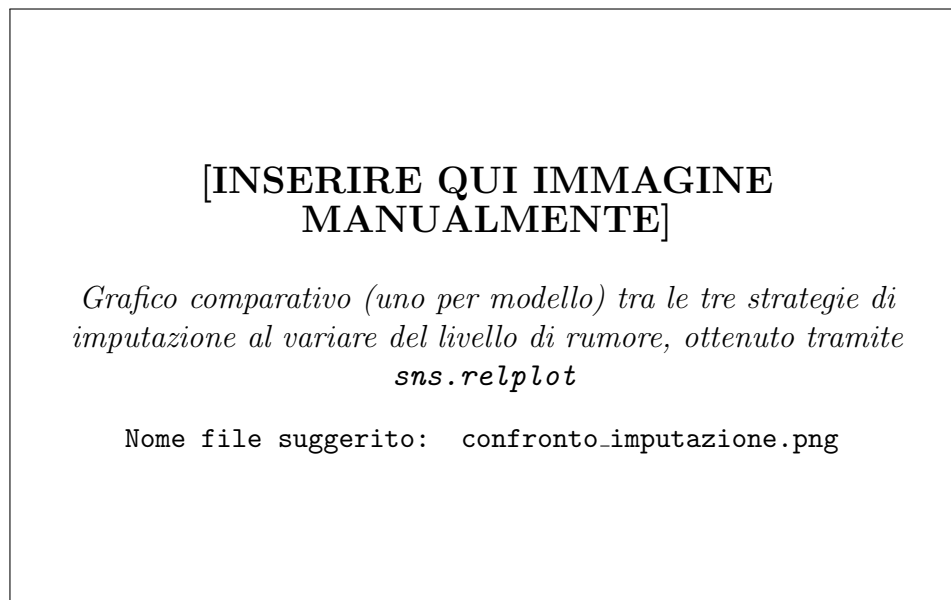


Figura 16: Confronto tra le strategie di imputazione dei valori mancanti (per modello)

9.2 Mitigazione dei valori duplicati

Sono state confrontate due strategie per la gestione dei duplicati:

- **Rimozione dei duplicati:** identificazione ed eliminazione delle righe identiche (`drop_duplicates()`), per evitare che il modello attribuisca peso eccessivo a osservazioni ripetute e per prevenire il *data leakage* in fase di validazione. Applicando questa tecnica, il training set torna, per tutti i livelli, alla dimensione originale di 735 righe (Tabella 14);
- **Aggregazione intelligente:** anziché rimuovere semplicemente le righe duplicate, si raggruppano le osservazioni identiche e si assegna come etichetta la *moda* del target tra le occorrenze duplicate, tecnica utile quando i duplicati presentano etichette discordanti (rumore di etichettatura).

Livello	Righe rimosse	Dimensione finale
Baseline (0%)	1	735
Low (15%)	111	735
Medium (40%)	295	735
High (55%)	405	735

Tabella 14: Effetto della rimozione dei duplicati sulla dimensione del training set.

Il grafico seguente confronta, per la SVM, la cross-validation "ottimistica" calcolata sui dati con duplicati (affetta da *leakage*) con quella "corretta" ottenuta dopo la rimozione e dopo l'aggregazione intelligente.

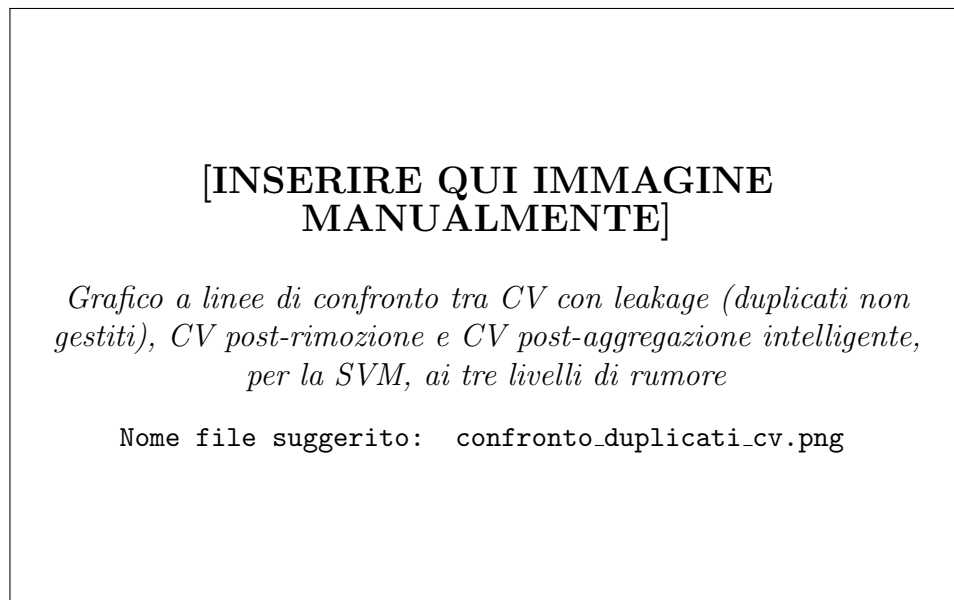


Figura 17: Confronto tra le strategie di gestione dei duplicati in Cross-Validation (SVM)

Come atteso, la CV calcolata sui dati con duplicati non gestiti sovrastima sistematicamente le reali prestazioni del modello; le strategie di rimozione e

aggregazione restituiscono invece stime più realistiche e coerenti con le prestazioni osservate sul test set pulito, dimostrando l'importanza di riconoscere e correggere il data leakage introdotto dai duplicati prima di ogni fase di validazione.

9.3 Mitigazione dei dati sporchi

Per contrastare l'effetto dei dati "sporchi" (rumore gaussiano e swap di valori binari), che si sono rivelati la tipologia di rumore più impattante, è stata adottata una pipeline di preprocessing robusto composta da:

- **RobustScaler**: normalizzazione basata su mediana e range interquartile, meno sensibile agli outlier rispetto allo **StandardScaler** classico;
- **PCA (Principal Component Analysis)**, mantenendo il numero minimo di componenti necessario a spiegare il **95%** della varianza totale, con l'obiettivo di filtrare parte del rumore concentrato nelle componenti a bassa varianza;
- modelli regolarizzati (Decision Tree con `max_depth=4`, `min_samples_split=20`, `min_samples_leaf=15`; SVM con $C = 0.1$).

Il numero di componenti principali necessarie per raggiungere il 95% della varianza spiegata, per ciascun livello di rumore, è riportato in Tabella 15.

Livello	Componenti principali (95% varianza)
Baseline (0%)	14
Low (15%)	16
Medium (40%)	14
High (55%)	16

Tabella 15: Numero di componenti principali necessarie per spiegare il 95% della varianza, per livello di rumore.

[INSERIRE QUI IMMAGINE
MANUALMENTE]

Grafico della varianza spiegata cumulativa in funzione del numero di componenti PCA, con soglia del 95% evidenziata, per ciascun livello di rumore (4 grafici, uno per livello)

Nome file suggerito: `pca_varianza_cumulativa.png`

Figura 18: Analisi PCA – Varianza cumulativa spiegata per livello di rumore

La Tabella 16 riporta l'accuracy media di cross-validation ottenuta con la pipeline robusta (RobustScaler + PCA + modelli regolarizzati), da confrontare con i valori di Tabella 12 ottenuti senza tecniche di mitigazione.

Livello	Decision Tree	SVM	Neural Network
Baseline (0%)	0.745	0.811	0.798
Low (15%)	0.726	0.769	0.750
Medium (40%)	0.669	0.698	0.704
High (55%)	0.689	0.708	0.702

Tabella 16: Accuracy media di CV (5-fold) con pipeline robusta (RobustScaler + PCA + regolarizzazione) sui dati sporchi.

Il confronto tra le due tabelle mostra come, per questa specifica tipologia di rumore, il beneficio della pipeline robusta risulti contenuto e non sempre univoco: ai livelli Low e High le prestazioni risultano leggermente superiori rispetto al caso non regolarizzato, mentre al livello Medium risultano sostanzialmente comparabili. Questo suggerisce che il rumore gaussiano applicato direttamente sulle feature, unito allo swap dei valori binari, alteri in modo diffuso la struttura informativa del dataset, rendendo difficile una sua completa mitigazione tramite le sole tecniche di preprocessing lineare (scaling e riduzione dimensionale).

10 Confronto finale tra le strategie

Per fornire una visione d'insieme, i risultati di tutte le strategie di gestione del rumore analizzate sono stati raccolti in un unico confronto grafico, organizzato per tipologia di rumore (valori mancanti con KNN Imputer, valori mancanti con Iterative Imputer, duplicati rimossi, duplicati aggregati, dati sporchi con

pipeline robusta) e per modello (Decision Tree, SVM, Neural Network), al variare del livello di rumore.

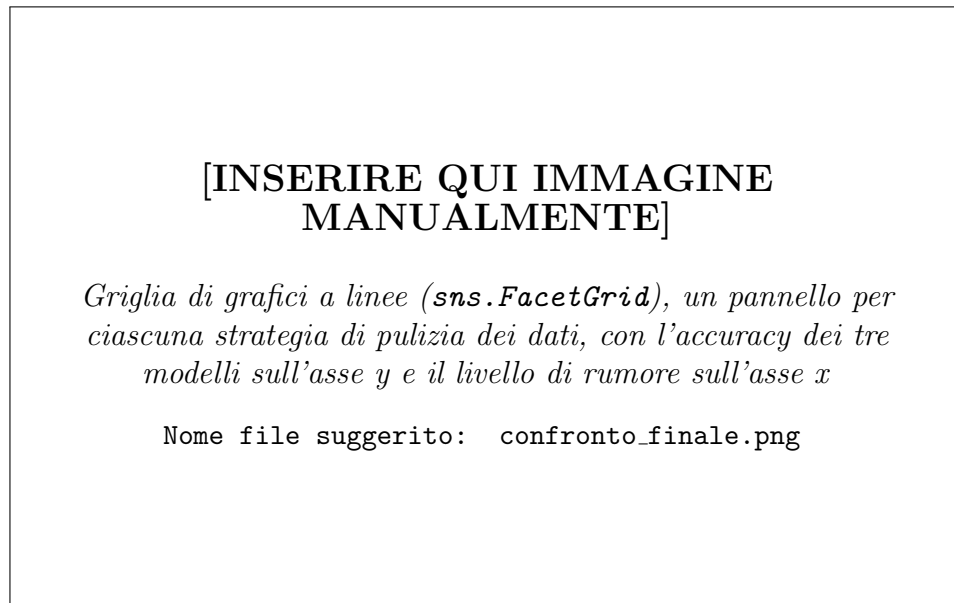


Figura 19: Confronto finale delle prestazioni dei modelli robusti per strategia di pulizia dei dati

Dall'analisi complessiva emergono le seguenti osservazioni:

- la **SVM** risulta, in generale, il modello più performante e più stabile al crescere del rumore, specialmente per i valori mancanti e i dati sporchi;
- i **valori mancanti** sono la tipologia di rumore più facilmente mitigabile, grazie a tecniche di imputazione relativamente mature (KNN, Iterative Imputer);
- i **duplicati**, se non correttamente identificati, non danneggiano le prestazioni reali del modello ma **falsano la stima** ottenuta tramite cross-validation (data leakage); una volta rimossi o aggregati correttamente, la valutazione torna realistica;
- i **dati sporchi** (rumore gaussiano e inversione di valori binari) costituiscono la sfida più difficile da mitigare, poiché alterano silenziosamente il contenuto informativo delle feature senza essere identificabili come nel caso dei valori mancanti.

11 Conclusioni

Il progetto ha permesso di analizzare in modo sistematico l'impatto di tre diverse tipologie di rumore (valori mancanti, duplicati, dati sporchi) sulle prestazioni di tre modelli di Machine Learning (Decision Tree, SVM, rete neurale), applicati a un problema reale di classificazione in ambito medicale.

I risultati mostrano che:

1. l'impatto del rumore **non è uniforme**: i valori mancanti causano un degrado moderato e ben gestibile tramite imputazione, i duplicati non peggiorano le prestazioni reali ma possono falsare la validazione se non gestiti correttamente, mentre i dati sporchi rappresentano la minaccia più seria e più difficile da correggere;
2. le tecniche di **regolarizzazione dei modelli** (pruning per il Decision Tree, soft margin per la SVM, dropout ed early stopping per la rete neurale) contribuiscono a contenere l'overfitting e, in generale, a rendere i modelli più stabili al crescere del rumore;
3. la **cross-validation** si conferma uno strumento fondamentale per la valutazione dei modelli, ma richiede attenzione: in presenza di duplicati non gestiti, può fornire stime eccessivamente ottimistiche a causa del data leakage;
4. non esiste un'unica tecnica di preprocessing "universale": la scelta della strategia più efficace dipende dalla natura specifica del rumore presente nei dati.

Come possibile sviluppo futuro, si potrebbero esplorare tecniche di rilevamento automatico del tipo di rumore presente nel dataset, così da selezionare in modo adattivo la strategia di preprocessing più adeguata, oltre a estendere l'analisi ad altri modelli (es. ensemble methods come Random Forest o Gradient Boosting) e ad altri dataset di dominio diverso, per verificare la generalizzabilità delle conclusioni ottenute.

A Elenco delle immagini da inserire manualmente

La Tabella 17 riassume, in ordine di comparizione nel documento, tutte le figure segnalate come "da inserire manualmente". Per ciascuna immagine è indicata la cella del notebook da cui è generata, così da poterla ritrovare ed esportare (ad esempio come screenshot o tramite `plt.savefig(...)` prima di `plt.show()`).

Nome file suggerito	Sezione	Contenuto
distribuzione_target.png	EDA	Countplot target binario
matrice_correlazione.png	EDA	Heatmap correlazione
istogrammi_variabili.png	EDA	Istogrammi variabili numeriche
cm_dt_baseline.png	Decision Tree	Confusion matrix (non potato)
albero_non_potato.png	Decision Tree	Plot albero non potato
cm_dt_pruned.png	Decision Tree	Confusion matrix (potato)
albero_potato.png	Decision Tree	Plot albero potato
nn_accuracy_epoche.png	Neural Network	Accuracy vs epoche
nn_loss_epoche.png	Neural Network	Loss vs epoche
cm_nn_baseline.png	Neural Network	Confusion matrix
cm_svm_baseline.png	SVM	Confusion matrix
confronto_accuracy_baseline.png	Confronto baseline	Barplot accuracy modelli
cm_nanlivelli.png	Impatto rumore	Confusion matrix per livello NaN
cm_duplivelli.png	Impatto rumore	Confusion matrix per livello duplicati
cm_dirtylivelli.png	Impatto rumore	Confusion matrix per livello dati sporchi
confronto_imputazione.png	Mitigazione NaN	Confronto strategie imputazione
confronto_duplicati_cv.png	Mitigazione duplicati	Confronto leakage/rimozione/agggregazione
pca_varianza_cumulativa.png	Mitigazione dati sporchi	Varianza cumulativa PCA
confronto_finale.png	Confronto finale	FacetGrid riepilogo generale

Tabella 17: Elenco riepilogativo delle immagini da inserire manualmente nel documento.